

Curso Java Backend - TechDesk

Contents

Curso Java Backend Passo a Passo — TechDesk (Arquitetura de Microsserviços)	1
Do fundamento de Java a dois microsserviços conversando entre si, de forma assíncrona, com frontend, testes, Docker e Git desde o dia 1	1
Como o curso está organizado	2
Módulo 0 — Preparando o Ambiente: Ferramentas, Git e Docker	2
Módulo 1 — Arquitetura do Projeto: Pacotes e Organização	5
Módulo 2 — Fundamentos de Java (Revisão Ativa)	6
Módulo 3 — Primeiro Microsserviço: techdesk-chamados-api	10
Módulo 4 — Modelagem e Persistência (JPA + MySQL no Docker)	12
Módulo 5 — CRUD REST Completo + Testes no Postman	14
Módulo 6 — Tratamento Global de Erros e Validação + Postman	18
Módulo 7 — Autenticação e Segurança (JWT) + Postman	19
Módulo 8 — Testes Automatizados (JUnit + Mockito)	22
Módulo 9 — Segundo Microsserviço: techdesk-notificacao-api	24
Módulo 10 — Comunicação Síncrona entre os Dois Serviços (REST)	27
Módulo 11 — Comunicação Assíncrona e Desacoplada (RabbitMQ)	29
Módulo 12 — Frontend Consumindo a API	33
Módulo 13 — Docker Compose Multi-Serviço (Aprofundamento)	36
Módulo 14 — Publicação, GitHub e Portfólio	38
Resumo do que este curso entrega	39

Curso Java Backend Passo a Passo — TechDesk (Arquitetura de Microsserviços)

Do fundamento de Java a dois microsserviços conversando entre si, de forma assíncrona, com frontend, testes, Docker e Git desde o dia 1

Este curso constrói o **TechDesk**, um sistema de gestão de chamados técnicos dividido em **dois microsserviços independentes**:

- **techdesk-chamados-api** — cadastra e gerencia os chamados técnicos (o serviço principal)
- **techdesk-notificacao-api** — recebe eventos do primeiro serviço e trata as notificações (o serviço consumidor)

Os dois primeiro se comunicam de forma síncrona (uma API chamando a outra via REST), para você sentir na pele o problema de acoplamento que isso gera — e depois você refatora essa comunicação para **assíncrona, via fila de mensagens (RabbitMQ)**, que é o padrão real usado no mercado para desacoplar serviços. Esse é o projeto de portfólio final: dois microsserviços reais, se comunicando de forma confiável e desacoplada, exatamente como se espera de um backend Java em 2026.

Como o curso está organizado

Cada módulo traz: objetivo, teoria (mais aprofundada nesta versão), **onde criar cada classe** (pacote exato), passo a passo com código comentado, um checkpoint detalhado do que você deve ter funcionando, e — a partir do CRUD — um roteiro de testes no **Postman**.

Módulo	Tema
0	Ambiente: ferramentas, Git e Docker desde o início
1	Arquitetura do projeto: pacotes e organização
2	Fundamentos de Java (revisão ativa)
3	Primeiro microsserviço: techdesk-chamados-api
4	Modelagem e persistência (JPA + MySQL no Docker)
5	CRUD REST completo + testes no Postman
6	Tratamento global de erros e validação + Postman
7	Autenticação e segurança (JWT) + Postman
8	Testes automatizados (JUnit + Mockito)
9	Segundo microsserviço: techdesk-notificacao-api
10	Comunicação síncrona entre os dois serviços (REST)
11	Comunicação assíncrona e desacoplada (RabbitMQ)
12	Frontend consumindo a API
13	Docker Compose multi-serviço (aprofundamento)
14	Publicação, GitHub e portfólio

Módulo 0 — Preparando o Ambiente: Ferramentas, Git e Docker

Objetivo

Instalar e confirmar cada ferramenta que o curso vai usar, e **só depois** inicializar o Git e subir a infraestrutura em Docker. A ordem importa: `docker-compose up` não roda numa máquina sem o Docker instalado — por isso este módulo tem duas fases bem separadas.

Teoria

Em um projeto profissional, o Git versiona o código desde o primeiro arquivo (mesmo vazio), e a infraestrutura (banco de dados, filas de mensagens) roda em containers desde o início — assim você tem sempre o mesmo ambiente, em qualquer máquina, sem o clássico “na minha máquina funciona”. Mas nada disso funciona sem as ferramentas de base instaladas primeiro. Por isso:

Fase 1 instala e verifica, Fase 2 usa.

Fase 1 — Instalar e verificar as ferramentas

Instale nesta ordem, e só passe para a próxima quando o comando de verificação funcionar:

1. Git — git-scm.com/downloads

```
git --version
```

2. JDK 21 (distribuição Temurin/Adoptium, gratuita)

```
java -version
```

3. Maven

```
mvn -version
```

4. IntelliJ IDEA Community (gratuito) ou **VS Code** com a extensão “Extension Pack for Java” — sem comando de verificação, só confirme que abre normalmente.

5. Docker Desktop (Windows/Mac) ou **Docker Engine + Docker Compose** (Linux) — [docker.com/get-started](https://docs.docker.com/get-started/). Este é o passo que faltava antes de qualquer docker-compose up funcionar.

```
docker --version
docker compose version
```

Abra o Docker Desktop (no Linux, confirme o serviço com `sudo systemctl status docker`) — ele precisa estar **aberto e rodando** sempre que você usar docker-compose no curso, não só instalado.

6. Postman — [postman.com/downloads](https://www.postman.com/downloads). Vamos usá-lo a partir do Módulo 5, mas instale agora.

7. DBeaver (ou MySQL Workbench) — cliente visual para olhar dentro do banco quando precisar depurar algo.

Só avance para a Fase 2 quando `git --version`, `java --version`, `mvn --version`, `docker --version` e `docker compose version` rodarem sem erro. Se algum falhar, o problema está na instalação — resolver isso agora evita erros confusos lá na frente.

Fase 2 — Estrutura do projeto, Git e infraestrutura Docker

1. Estrutura de pastas do projeto (monorepo simples). Escolha uma pasta de trabalho no seu computador e, com um terminal aberto **dentro dela**, rode:

```
mkdir techdesk && cd techdesk
```

A partir daqui, **todo comando deste módulo roda dentro de techdesk/**, salvo aviso contrário. Estrutura final:

```
techdesk/
├── techdesk-chamados-api/      ← terminal fica aqui na maior parte do curso
│                               # microsserviço 1 (Módulo 3 em diante)
├── techdesk-notificacao-api/   # microsserviço 2 (Módulo 9 em diante)
├── frontend/                  # Módulo 12
├── docker-compose.yml         # infraestrutura compartilhada
└── README.md
```

2. Inicialize o Git (ainda dentro de techdesk/):

```
git init
echo "# TechDesk – Sistema de Gestão de Chamados Técnicos (Microsserviços)" > README.md
```

Crie um `.gitignore` (padrão Java/Maven/IDE):

```
target/
*.class
.idea/
*.iml
.vscode/
.env
```

3. Crie o repositório no GitHub (pelo site, vazio, sem README, para não gerar conflito) e conecte, no mesmo terminal:

```
git remote add origin https://github.com/SEU_USUARIO/techdesk.git
git add .
git commit -m "chore: estrutura inicial do projeto"
git push -u origin main
```

A partir de agora, **cada módulo termina com um commit**. Isso vira, sozinho, um histórico de commits organizado no GitHub — outra coisa que pesa positivamente numa avaliação técnica.

4. Só agora, com o Docker Desktop aberto e rodando, suba a infraestrutura base — banco de dados e fila de mensagens, mesmo sem nenhuma aplicação Java ainda. Crie o arquivo abaixo na raiz de techdesk/:

```
# techdesk/docker-compose.yml
version: '3.8'

services:
  mysql:
    image: mysql:8.0
    container_name: techdesk-mysql
    environment:
      MYSQL_ROOT_PASSWORD: techdesk123
      MYSQL_DATABASE: techdesk_chamados
    ports:
      - "3306:3306"
    volumes:
      - mysql-data:/var/lib/mysql

  rabbitmq:
    image: rabbitmq:3-management
    container_name: techdesk-rabbitmq
    ports:
      - "5672:5672" # porta usada pelas aplicações
      - "15672:15672" # painel visual (http://localhost:15672, login: guest/guest)

volumes:
  mysql-data:
```

No terminal, **dentro de techdesk/** (onde está o `docker-compose.yml` que você acabou de criar):
`docker-compose up -d`

Se aparecer erro do tipo “Cannot connect to the Docker daemon”, o Docker Desktop não está aberto — abra o aplicativo, espere ele terminar de iniciar, e rode o comando de novo.

Acesse `http://localhost:15672` no navegador (usuário `guest`, senha `guest`) e confirme que o painel do RabbitMQ abre — vamos usá-lo de verdade no Módulo 11, mas já fica rodando desde agora.

```
git add docker-compose.yml
git commit -m "chore: infraestrutura base (MySQL + RabbitMQ) via Docker Compose"
git push
```

Checkpoint — o que você deve ter, e como confirmar

- Os cinco comandos de verificação da Fase 1 rodam sem erro.
- `git log` (dentro de `techdesk/`) mostra pelo menos 2 commits, e o repositório aparece no seu GitHub com esses commits.
- `docker ps` lista dois containers rodando: `techdesk-mysql` e `techdesk-rabbitmq`, ambos com status Up.
- O painel do RabbitMQ abre no navegador em `localhost:15672`.
- Se qualquer um desses itens falhar, pare aqui e resolva antes de avançar — todos os módulos seguintes dependem dessa base.

Módulo 1 — Arquitetura do Projeto: Pacotes e Organização

Objetivo

Entender exatamente onde cada tipo de classe deve morar dentro do projeto, antes de criar a primeira — para nunca ficar em dúvida sobre “isso é service ou controller?”.

Teoria (aprofundada)

Existem duas formas comuns de organizar pacotes em um projeto Spring Boot:

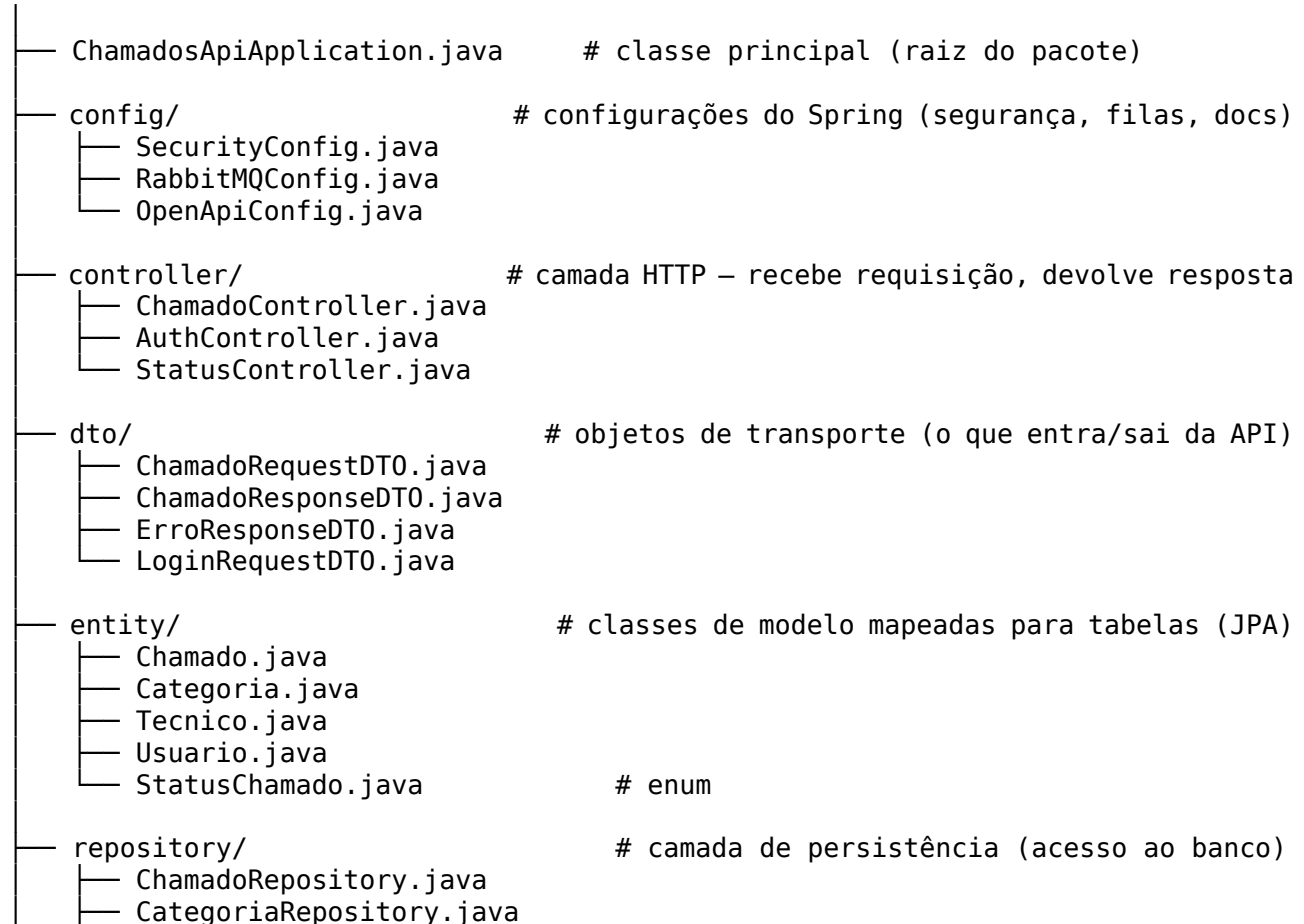
Package by layer (por camada) — agrupa por *o que a classe é* (todos os controllers juntos, todos os services juntos). É a mais usada em projetos pequenos/médios e a mais didática para entender a arquitetura em camadas. **É a que vamos usar neste curso.**

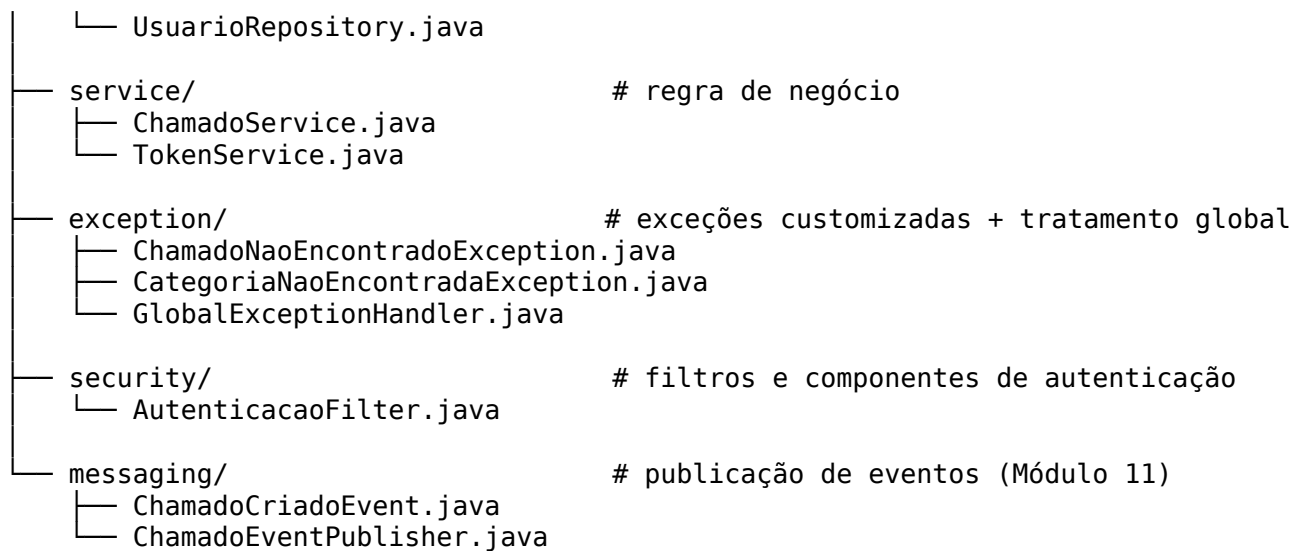
Package by feature (por funcionalidade) — agrupa por *o que a classe faz* (tudo relacionado a “chamados” num pacote só: chamado.ChamadoController, chamado.ChamadoService etc.). Escala melhor em projetos grandes, porque reduz o acoplamento entre módulos de negócio diferentes. Vale conhecer o nome e o motivo, mesmo usando a outra abordagem agora — é uma pergunta comum em entrevista de nível pleno.

A regra de ouro, independente da abordagem: **cada classe tem uma responsabilidade só** (Single Responsibility Principle). Um Controller nunca acessa o banco diretamente; um Service nunca sabe que existe HTTP; um Repository nunca contém regra de negócio.

Estrutura de pacotes do techdesk-chamados-api

com.techdesk.chamados





O `techdesk-notificacao-api` (Módulo 9) segue exatamente o mesmo padrão de pacotes, só que menor: `config`, `controller`, `dto`, `entity`, `repository`, `service`, `messaging`.

Regra prática que vamos seguir o curso inteiro: toda vez que uma nova classe for criada, o código virá com um comentário na primeira linha indicando o caminho completo, assim:

```
// src/main/java/com/techdesk/chamados/entity/Chamado.java
```

Checkpoint

Você consegue, sem olhar a tabela acima, dizer de cabeça em qual pacote entraria uma nova classe `RelatorioService` (resposta: `service/`) e uma nova classe `ChamadoNaoPertenceAoUsuarioException` (resposta: `exception/`).

Módulo 2 — Fundamentos de Java (Revisão Ativa)

Objetivo

Reativar os quatro pilares mais cobrados em entrevista — Orientação a Objetos, Coleções, Generics e Exceções — com exercícios isolados, antes de aplicá-los no projeto real.

Teoria (aprofundada)

Java é uma linguagem **fortemente tipada e orientada a objetos por padrão**: praticamente tudo é ou faz parte de uma classe. Isso tem um efeito direto no dia a dia: o compilador pega muitos erros antes mesmo do código rodar, o que é uma vantagem grande sobre linguagens dinâmicas como Python quando o time cresce e o projeto precisa de previsibilidade — um dos motivos pelos quais bancos e sistemas críticos preferem Java. Os quatro pilares da Orientação a Objetos (**encapsulamento, herança, polimorfismo e abstração**) existem para controlar a complexidade à medida que o projeto cresce, evitando que uma mudança em um lugar quebre o sistema inteiro em cascata.

Este módulo é só para prática isolada — crie uma pasta separada `java-fundamentos-exercicios/` fora do projeto principal, sem Spring, só para rodar estes exemplos com `java NomeDaClasse.java`.

2.1 Orientação a Objetos

```
// Classe que representa um Técnico do helpdesk.
// Encapsulamento: os atributos são privados (private), só acessíveis
// de fora da classe através de métodos públicos (getters/setters).
// Isso protege o estado interno do objeto contra alterações indevidas.
public class Tecnico {
    private String nome;
    private String especialidade;

    // Construtor: define como um objeto Tecnico nasce.
    // Java não permite criar um Tecnico "pela metade" se o construtor exigir os dados.
    public Tecnico(String nome, String especialidade) {
        this.nome = nome;
        this.especialidade = especialidade;
    }

    public String getNome() {
        return nome;
    }

    public String getEspecialidade() {
        return especialidade;
    }

    // Método de instância: representa um comportamento do objeto.
    public String apresentar() {
        return "Técnico " + nome + " - especialista em " + especialidade;
    }
}
```

```
// Herança: TecnicoSenior "é um" Tecnico, mas com um comportamento extra.
// "extends" reaproveita atributos e métodos da classe pai (reuso de código).
public class TecnicoSenior extends Tecnico {
    private int anosExperiencia;

    public TecnicoSenior(String nome, String especialidade, int anosExperiencia) {
        super(nome, especialidade); // chama o construtor da classe pai primeiro
        this.anosExperiencia = anosExperiencia;
    }

    // Polimorfismo: sobrescrevemos (@Override) o comportamento herdado.
    // Quem chama apresentar() não precisa saber se é um Tecnico comum ou
    // um TecnicoSenior – o comportamento certo é escolhido em tempo de execução.
    @Override
    public String apresentar() {
        return super.apresentar() + " (" + anosExperiencia + " anos de experiência)";
    }
}
```

```
// Interface: um contrato – define O QUE uma classe deve fazer, não COMO.
// No projeto real, os Repositories do Spring Data JPA são interfaces (Módulo 4).
public interface Notificavel {
    void notificar(String mensagem);
}
```

```
// Abstração: TecnicoNotificavel implementa o contrato à sua maneira.
public class TecnicoNotificavel extends Tecnico implements Notificavel {
    public TecnicoNotificavel(String nome, String especialidade) {
        super(nome, especialidade);
    }
}
```

```

    }

    @Override
    public void notificar(String mensagem) {
        System.out.println("Notificando " + getNome() + ": " + mensagem);
    }
}

```

2.2 Coleções e Generics

```

import java.util.*;

public class ExemploColecoes {
    public static void main(String[] args) {

        // List: coleção ORDENADA, permite elementos duplicados.
        // Use quando a ordem de inserção importa.
        List<String> categorias = new ArrayList<>();
        categorias.add("Hardware");
        categorias.add("Rede");
        categorias.add("Software");

        // Set: coleção SEM duplicados. Use quando só importa "existe ou não existe".
        Set<String> tecnicosUnicos = new HashSet<>();
        tecnicosUnicos.add("Maria");
        tecnicosUnicos.add("Maria"); // ignorado silenciosamente, já existe

        // Map: pares chave-valor. Ideal para contagens e buscas rápidas por chave.
        Map<String, Integer> chamadosPorStatus = new HashMap<>();
        chamadosPorStatus.put("ABERTO", 5);
        chamadosPorStatus.put("RESOLVIDO", 12);

        // O <String, Integer> acima é Generics em ação: o compilador garante
        // que só Strings entram como chave e só Integers como valor –
        // erro de tipo vira erro de COMPILAÇÃO, não de execução (muito mais barato de
        // ↳ corrigir).

        for (Map.Entry<String, Integer> entry : chamadosPorStatus.entrySet()) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }

        // Stream API (muito cobrado em entrevista pleno): processa coleções
        // de forma declarativa, em vez de loops manuais.
        List<String> categoriasComMaisDeCincoLetras = categorias.stream()
            .filter(c -> c.length() > 5) // mantém só o que passa na condição
            .map(String::toUpperCase)    // transforma cada elemento
            .toList();                   // materializa o resultado numa nova List

        System.out.println(categoriasComMaisDeCincoLetras);
    }
}

// Generics em uma classe própria: uma "caixa" reaproveitável para qualquer tipo,
// mantendo segurança de tipo sem precisar duplicar código nem fazer casting manual.
public class Resposta<T> {
    private T dado;
    private boolean sucesso;
}

```



```

public Resposta(T dado, boolean sucesso) {
    this.dado = dado;
    this.sucesso = sucesso;
}

public T getDado() {
    return dado;
}

public boolean isSucesso() {
    return sucesso;
}
}
// Uso: Resposta<Chamado> ou Resposta<List<Tecnico>> – o MESMO código de Resposta<T>
// serve para qualquer tipo. É esse reuso com segurança que Generics resolve.

```

2.3 Tratamento de Exceções

```

// Exceção customizada: no projeto real (Módulo 6), isso é lançado quando
// alguém busca um chamado que não existe, e vira automaticamente um HTTP 404.
public class ChamadoNaoEncontradoException extends RuntimeException {
    public ChamadoNaoEncontradoException(Long id) {
        super("Chamado com id " + id + " não encontrado.");
    }
}

```

```

public class ExemploExcecoes {

    public static Chamado buscarChamado(Long id, Map<Long, Chamado> banco) {
        // Boa prática: validar e lançar uma exceção ESPECÍFICA e com contexto,
        // em vez de deixar o erro "estourar" sem explicação
        // (o que aconteceria com um NullPointerException genérico).
        if (!banco.containsKey(id)) {
            throw new ChamadoNaoEncontradoException(id);
        }
        return banco.get(id);
    }

    public static void main(String[] args) {
        Map<Long, Chamado> banco = new HashMap<>();
        try {
            Chamado c = buscarChamado(99L, banco);
        } catch (ChamadoNaoEncontradoException e) {
            // Tratamento específico e legível
            System.out.println("Erro tratado: " + e.getMessage());
        } finally {
            // finally executa sempre, com ou sem erro – útil para liberar
            // recursos como conexões de banco ou arquivos abertos.
            System.out.println("Consulta finalizada.");
        }
    }
}

```

Checkpoint — o que você deve conseguir fazer

- Rodar os três exemplos (ExemploColecoes, Resposta, ExemploExcecoes) sem erro de compilação.

- Explicar em voz alta, sem consultar nada: a diferença entre List/Set/Map (e quando usar cada um), o que Generics evita, e por que criamos ChamadoNaoEncontradoException em vez de deixar um erro genérico estourar.
- Se travar em algum ponto, esse é exatamente o sinal de qual conceito revisar antes de seguir — não pule para o Módulo 3 sem esse checkpoint validado.

Módulo 3 — Primeiro Microserviço: techdesk-chamados-api

Objetivo

Criar o projeto Spring Boot do primeiro microserviço e subir o primeiro endpoint.

Teoria (aprofundada)

Spring Boot reduz a configuração manual do Spring “puro” através de **auto-configuração** e **starters** (pacotes prontos de dependências). O conceito mais importante para internalizar agora é **Inversão de Controle (IoC)** com **Injeção de Dependência (DI)**: em vez de uma classe criar (new) suas próprias dependências, ela declara do que precisa e o **container do Spring** entrega essa dependência pronta. Isso desacopla as classes entre si — e é exatamente esse desacoplamento que torna possível substituir uma dependência real por um mock nos testes (Módulo 8), ou trocar a implementação sem reescrever quem a usa.

Passo a passo

1. Acesse start.spring.io.
2. Configure: **Maven, Java 21, Spring Boot 3.x**, artifactId: techdesk-chamados-api, pacote base com.techdesk.chamados.
3. Dependências **só as essenciais por enquanto**: Spring Web, Validation, Spring Boot DevTools.

Por que só essas três agora? Se Spring Data JPA e MySQL Driver entrarem já, o Spring Boot tenta configurar uma conexão de banco assim que a aplicação sobe — e como o application.yml com os dados do banco só existe no Módulo 4, a aplicação falharia ao iniciar. O mesmo vale para Spring Security (bloquearia todos os endpoints, inclusive o /api/status, antes de existir qualquer regra configurada no Módulo 7) e Spring AMQP (só faz sentido a partir do Módulo 11). A regra do curso: **cada dependência entra no módulo em que é usada**, nunca antes — isso evita o clássico “a aplicação não sobe” por causa de uma configuração pela metade.

4. Extraia o .zip de forma que a pasta gerada fique em techdesk/techdesk-chamados-api/ (ou seja, dentro da pasta techdesk/ que você criou no Módulo 0).

```
// techdesk-chamados-api/src/main/java/com/techdesk/chamados/ChamadosApiApplication.java

// Classe principal – ponto de entrada da aplicação.
// @SpringBootApplication ativa auto-configuração, component scan e configuração do
//   ↪ Spring Boot.
@SpringBootApplication
public class ChamadosApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(ChamadosApiApplication.class, args);
    }
}
```

```
// techdesk-chamados-
↪ api/src/main/java/com/techdesk/chamados/controller/StatusController.java

@RestController // diz ao Spring: esta classe responde requisições HTTP com JSON/texto no
↪ corpo
@RequestMapping("/api/status")
public class StatusController {

    @GetMapping
    public String status() {
        return "techdesk-chamados-api no ar!";
    }
}
```

- Abra um terminal **dentro de techdesk/techdesk-chamados-api/** (é essa pasta específica, não a raiz techdesk/) e rode:

```
mvn spring-boot:run
```

(ou use o botão “Run” da IDE com esse projeto aberto). Acesse <http://localhost:8080/api/status> no navegador — esse terminal fica aberto e rodando; vamos chamá-lo de **Terminal 1** pelo resto do curso.

Injeção de Dependência na prática

```
// SEM DI (ruim para testar – acoplamento forte, ChamadoController
// nunca pode ser testado sem instanciar um ChamadoService de verdade):
public class ChamadoControllerRuim {
    private ChamadoService service = new ChamadoService();
}

// COM DI (jeito Spring – testável e desacoplado):
@RestController
public class ChamadoController {

    private final ChamadoService service;

    // O Spring injeta automaticamente uma instância de ChamadoService aqui,
    // via "constructor injection" (a forma recomendada hoje, em vez de @Autowired em
    ↪ campo).
    public ChamadoController(ChamadoService service) {
        this.service = service;
    }
}
```

```
git add .
git commit -m "feat(chamados-api): estrutura inicial do projeto Spring Boot"
git push
```

Checkpoint — o que você deve ter, e como confirmar

- A aplicação sobe sem erro no console (procure a mensagem Started ChamadosApiApplication).
- <http://localhost:8080/api/status> responde "techdesk-chamados-api no ar!" no navegador.
- Você sabe explicar, com um exemplo próprio (fora do projeto), a diferença entre criar uma dependência com new e recebê-la injetada pelo Spring.

Módulo 4 — Modelagem e Persistência (JPA + MySQL no Docker)

Objetivo

Criar as entidades do domínio e conectar ao MySQL que já está rodando via Docker desde o Módulo 0.

Teoria (aprofundada)

JPA (Jakarta Persistence API) é a especificação Java para mapear objetos para tabelas relacionais — o padrão conhecido como ORM (Object-Relational Mapping). O **Hibernate** é a implementação que o Spring Data JPA usa por baixo. Isso não substitui seu conhecimento de SQL — substitui a necessidade de *escrever* SQL repetitivo à mão para operações simples. Você continua precisando entender o SQL gerado (por isso vamos deixar `show-sql: true` ligado), principalmente para depurar performance depois.

Passo 1 — Adicionar as dependências de persistência

O projeto do Módulo 3 só tinha Spring Web, Validation e DevTools. Agora que vamos usar banco de dados, **pare a aplicação** (Ctrl+C no Terminal 1, se ainda estiver rodando) e adicione ao `pom.xml` do `techdesk-chamados-api` (abra o arquivo na raiz desse projeto, não na raiz de `techdesk/`):

```
<!-- techdesk-chamados-api/pom.xml (dentro de <dependencies>) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```

Salve o arquivo — a IDE costuma reimportar as dependências automaticamente (procure o ícone de “reload Maven”). Se preferir garantir pela linha de comando, abra um terminal **dentro de** `techdesk/techdesk-chamados-api/` e rode:

```
mvn clean install
```

Não rode a aplicação ainda — sem o `application.yml` configurado (próximo passo), ela vai falhar ao tentar conectar num banco que não foi informado.

Passo 2 — Modelo de domínio

- **Usuario** — quem abre o chamado
- **Tecnico** — quem resolve o chamado
- **Categoria** — tipo do chamado (Hardware, Rede, Software...)
- **Chamado** — o ticket, com status (ABERTO, EM_ANDAMENTO, RESOLVIDO, FECHADO)

```
// techdesk-chamados-api/src/main/java/com/techdesk/chamados/entity/StatusChamado.java

// Enum: representa um conjunto FIXO e finito de valores possíveis.
// Mais seguro que usar String solta — o compilador impede um status inválido.
public enum StatusChamado {
```

```

        ABERTO, EM_ANDAMENTO, RESOLVIDO, FECHADO
    }

// techdesk-chamados-api/src/main/java/com/techdesk/chamados/entity/Categoria.java

@Entity // diz ao JPA: esta classe vira uma tabela
@Table(name = "categorias")
public class Categoria {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // auto-incremento no banco
    private Long id;

    @Column(nullable = false, unique = true)
    private String nome;

    public Categoria() {} // construtor vazio exigido pelo JPA

    // getters e setters – gere pela IDE (Alt+Insert no IntelliJ / botão direito no VS
    ↪ Code)
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
}

// techdesk-chamados-api/src/main/java/com/techdesk/chamados/entity/Chamado.java

@Entity
@Table(name = "chamados")
public class Chamado {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String titulo;

    @Column(length = 1000)
    private String descricao;

    @Enumerated(EnumType.STRING) // salva o enum como texto legível ("ABERTO"), não como
    ↪ número
    private StatusChamado status;

    private LocalDateTime dataAbertura;
    private LocalDateTime dataFechamento;

    // Relacionamento N:1 – vários chamados pertencem a UMA categoria
    @ManyToOne
    @JoinColumn(name = "categoria_id")
    private Categoria categoria;

    // Relacionamento N:1 – vários chamados podem ser atendidos por UM técnico
    @ManyToOne
    @JoinColumn(name = "tecnico_id")
    private Tecnico tecnico;

    @ManyToOne

```

```

@JoinColumn(name = "usuario_id", nullable = false)
private Usuario usuario;

public Chamado() {}

// getters e setters – gere pela IDE; são muitos, mas mecânicos.
// Dica: no IntelliJ, Alt+Insert > Getter and Setter > selecione todos os campos.
}

```

Passo 3 — Conectando ao MySQL do Docker

```

# techdesk-chamados-api/src/main/resources/application.yml
spring:
  application:
    name: techdesk-chamados-api
  datasource:
    url: jdbc:mysql://localhost:3306/techdesk_chamados
    username: root
    password: techdesk123
  jpa:
    hibernate:
      ddl-auto: update # Hibernate cria/atualiza as tabelas automaticamente a partir das
↪ @Entity
      show-sql: true # mostra o SQL gerado no console – essencial para aprender e
↪ depurar
server:
  port: 8080

```

O banco techdesk_chamados já existe porque foi criado pela variável MYSQL_DATABASE no docker-compose.yml do Módulo 0 — não precisa criar manualmente. Confirme antes que o container techdesk-mysql está Up (docker ps); se não estiver, volte para a pasta techdesk/ e rode docker-compose up -d.

Agora sim: no **Terminal 1** (dentro de techdesk/techdesk-chamados-api/), rode mvn spring-boot:run novamente.

```

git add .
git commit -m "feat(chamados-api): modelagem de entidades e conexão com MySQL via Docker"
git push

```

Checkpoint — o que você deve ter, e como confirmar

- Ao rodar a aplicação, o console mostra comandos create table ou alter table no log (por causa do show-sql: true).
- Abrindo o MySQL (via DBeaver, ou docker exec -it techdesk-mysql mysql -uroot -p), as tabelas chamados, categorias, tecnicos e usuarios existem, mesmo vazias.
- Você sabe explicar a diferença entre @ManyToOne e @OneToMany usando o exemplo Chamado/Categoria.

Módulo 5 — CRUD REST Completo + Testes no Postman

Objetivo

Implementar criar, listar, buscar, atualizar status e excluir chamados — e validar cada endpoint no Postman antes de seguir.

Teoria (aprofundada)

A arquitetura em camadas existe para isolar responsabilidades: o **Controller** só entende HTTP (requisição/resposta), o **Service** só entende regra de negócio, o **Repository** só entende persistência. Uma mudança de regra de negócio nunca deveria exigir alterar um Controller; uma mudança de banco de dados nunca deveria exigir alterar um Service. Essa separação é o que torna o projeto testável (Módulo 8) e sustentável conforme cresce. Além disso, nunca expomos a @Entity diretamente na API — usamos **DTOs** para controlar exatamente o que entra e sai, evitando vaziar campos internos (como senha) ou criar acoplamento entre o formato do banco e o formato da API pública.

```
// techdesk-chamados-
↪ api/src/main/java/com/techdesk/chamados/repository/ChamadoRepository.java

// Repository: o Spring Data JPA gera as queries automaticamente a partir
// da ASSINATURA do método — não escrevemos SQL manual aqui.
public interface ChamadoRepository extends JpaRepository<Chamado, Long> {
    List<Chamado> findByStatus(StatusChamado status);
    List<Chamado> findByTecnicoId(Long tecnicoId);
}

// techdesk-chamados-api/src/main/java/com/techdesk/chamados/dto/ChamadoRequestDTO.java

// record: forma enxuta do Java para uma classe imutável, ideal para DTOs.
// As anotações de validação (@NotBlank, @NotNull) são checadas automaticamente
// pelo Spring quando o DTO chega no Controller com @Valid.
public record ChamadoRequestDTO(
    @NotBlank String titulo,
    @NotBlank String descricao,
    @NotNull Long categoriaId
) {}

// techdesk-chamados-api/src/main/java/com/techdesk/chamados/dto/ChamadoResponseDTO.java

public record ChamadoResponseDTO(
    Long id,
    String titulo,
    String descricao,
    StatusChamado status,
    String categoriaNome,
    LocalDateTime dataAbertura
) {}

// techdesk-chamados-api/src/main/java/com/techdesk/chamados/service/ChamadoService.java

@Service // camada de regra de negócio — o Spring gerencia o ciclo de vida desse bean
public class ChamadoService {

    private final ChamadoRepository chamadoRepository;
    private final CategoriaRepository categoriaRepository;

    public ChamadoService(ChamadoRepository chamadoRepository, CategoriaRepository
        ↪ categoriaRepository) {
        this.chamadoRepository = chamadoRepository;
        this.categoriaRepository = categoriaRepository;
    }

    public ChamadoResponseDTO criar(ChamadoRequestDTO dto, Usuario usuarioLogado) {
        Categoria categoria = categoriaRepository.findById(dto.categoriaId())
            .orElseThrow(() -> new CategoriaNaoEncontradaException(dto.categoriaId()));
```

```

        Chamado chamado = new Chamado();
        chamado.setTitulo(dto.titulo());
        chamado.setDescricao(dto.descricao());
        chamado.setCategoria(categoria);
        chamado.setStatus(StatusChamado.ABERTO);
        chamado.setDataAbertura(LocalDateTime.now());
        chamado.setUsuario(usuarioLogado);

        Chamado salvo = chamadoRepository.save(chamado);
        return toResponseDTO(salvo);
    }

    public List<ChamadoResponseDTO> listarTodos() {
        return chamadoRepository.findAll().stream()
            .map(this::toResponseDTO)
            .toList();
    }

    public ChamadoResponseDTO buscarPorId(Long id) {
        Chamado chamado = chamadoRepository.findById(id)
            .orElseThrow(() -> new ChamadoNaoEncontradoException(id));
        return toResponseDTO(chamado);
    }

    public ChamadoResponseDTO atualizarStatus(Long id, StatusChamado novoStatus) {
        Chamado chamado = chamadoRepository.findById(id)
            .orElseThrow(() -> new ChamadoNaoEncontradoException(id));
        chamado.setStatus(novoStatus);
        if (novoStatus == StatusChamado.FECHADO) {
            chamado.setDataFechamento(LocalDateTime.now());
        }
        return toResponseDTO(chamadoRepository.save(chamado));
    }

    public void excluir(Long id) {
        if (!chamadoRepository.existsById(id)) {
            throw new ChamadoNaoEncontradoException(id);
        }
        chamadoRepository.deleteById(id);
    }

    // Método auxiliar privado: converte Entity em DTO – mantém essa conversão
    // num único lugar, em vez de espalhar pelo Controller.
    private ChamadoResponseDTO toResponseDTO(Chamado c) {
        return new ChamadoResponseDTO(
            c.getId(), c.getTitulo(), c.getDescricao(),
            c.getStatus(), c.getCategoria().getNome(), c.getDataAbertura()
        );
    }
}

```

```

// techdesk-chamados-
↪ api/src/main/java/com/techdesk/chamados/controller/ChamadoController.java

```

```

@RestController
@RequestMapping("/api/chamados")
public class ChamadoController {

```



```

private final ChamadoService service;

public ChamadoController(ChamadoService service) {
    this.service = service;
}

@PostMapping
public ResponseEntity<ChamadoResponseDTO> criar(@Valid @RequestBody ChamadoRequestDTO
    ↪ dto,
                                           @AuthenticationPrincipal Usuario
                                           ↪ usuarioLogado) {
    ChamadoResponseDTO criado = service.criar(dto, usuarioLogado);
    return ResponseEntity.status(HttpStatus.CREATED).body(criado); // 201 Created
}

@GetMapping
public ResponseEntity<List<ChamadoResponseDTO>> listar() {
    return ResponseEntity.ok(service.listarTodos()); // 200 OK
}

@GetMapping("/{id}")
public ResponseEntity<ChamadoResponseDTO> buscar(@PathVariable Long id) {
    return ResponseEntity.ok(service.buscarPorId(id));
}

@PatchMapping("/{id}/status")
public ResponseEntity<ChamadoResponseDTO> atualizarStatus(@PathVariable Long id,
    @RequestParam StatusChamado
    ↪ status) {
    return ResponseEntity.ok(service.atualizarStatus(id, status));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> excluir(@PathVariable Long id) {
    service.excluir(id);
    return ResponseEntity.noContent().build(); // 204 No Content
}
}

```

Testes no Postman

Pré-requisito: techdesk-chamados-api precisa estar rodando no **Terminal 1** (mvn spring-boot:run dentro de techdesk/techdesk-chamados-api/), e o container techdesk-mysql precisa estar Up. Abra o Postman (instalado no Módulo 0) e crie uma **Collection** chamada TechDesk, com uma pasta Chamados dentro dela. Para cada requisição abaixo: clique em “New” → “HTTP Request” dentro da pasta, confira o método, a URL, o corpo (aba Body → raw → JSON) e o status esperado.

- 1. Criar categoria (pré-requisito, direto no banco ou endpoint simples de Categoria — crie um CategoriaController equivalente ao ChamadoController, só com criar e listar).**
- 2. Criar chamado** - Método: POST - URL: http://localhost:8080/api/chamados - Body (raw JSON):

```

{
  "titulo": "Impressora não liga",
  "descricao": "Testado o cabo de energia, sem sucesso",
  "categoriaId": 1
}

```

- Esperado: status 201 Created, resposta com id, status: "ABERTO" e dataAbertura preenchida. Guarde o id retornado — vamos usar nas próximas chamadas.

3. Listar chamados - Método: GET - URL: `http://localhost:8080/api/chamados` - Esperado: status 200 OK, um array JSON contendo o chamado criado no passo anterior.

4. Buscar chamado por id - Método: GET - URL: `http://localhost:8080/api/chamados/1` (troque 1 pelo id real) - Esperado: 200 OK com os dados do chamado específico.

5. Atualizar status - Método: PATCH - URL: `http://localhost:8080/api/chamados/1/status?status=EM_A` - Esperado: 200 OK, e no corpo da resposta status: "EM_ANDAMENTO".

6. Excluir chamado - Método: DELETE - URL: `http://localhost:8080/api/chamados/1` - Esperado: status 204 No Content, sem corpo de resposta. Repetir o GET do passo 4 deve agora retornar erro (tratado no Módulo 6).

Salve a collection e exporte (Export → Collection v2.1) — isso também entra no repositório GitHub, numa pasta postman/.

```
git add postman/
git commit -m "feat(chamados-api): CRUD completo de chamados + collection Postman"
git push
```

Checkpoint — o que você deve ter, e como confirmar

- As 6 requisições do Postman acima retornam exatamente os status codes descritos, na ordem.
- Você entende por que POST retorna 201 (recurso criado) enquanto DELETE retorna 204 (sucesso, sem conteúdo para devolver).
- A collection do Postman está exportada e versionada no Git.

Módulo 6 — Tratamento Global de Erros e Validação + Postman

Objetivo

Centralizar o tratamento de exceções para que a API sempre devolva erros padronizados, em vez de um HTTP 500 genérico.

Teoria (aprofundada)

Sem tratamento global, qualquer exceção não capturada vira um HTTP 500 com uma stack trace exposta — isso é falha de segurança (vaza detalhes internos) e péssima experiência para quem consome a API. `@RestControllerAdvice` funciona como um “interceptor global”: qualquer exceção lançada em qualquer Controller passa primeiro por aqui antes de virar resposta HTTP. Essa é a diferença entre uma API amadora e uma API pronta para produção — e é exatamente o tipo de detalhe que entrevistadores técnicos perguntam.

```
// techdesk-chamados-api/src/main/java/com/techdesk/chamados/dto/ErroResponseDTO.java
public record ErroResponseDTO(String mensagem, int status, LocalDateTime timestamp) {}
```

```
// techdesk-chamados-api/src/main/java/com/techdesk/chamados/exception/GlobalExceptionHandler.java
```

```
@RestControllerAdvice // intercepta exceções lançadas por QUALQUER Controller da aplicação
public class GlobalExceptionHandler {
```

```
    @ExceptionHandler(ChamadoNaoEncontradoException.class)
```

```
    public ResponseEntity<ErroResponseDTO>
```

```
        handleChamadoNaoEncontrado(ChamadoNaoEncontradoException ex) {
```

```

        ErrorResponseDTO erro = new ErrorResponseDTO(ex.getMessage(), 404,
↪      LocalDateTime.now());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(erro);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponseDTO>
    ↪      handleValidacao(MethodArgumentNotValidException ex) {
        // Junta todos os erros de campo (ex: "titulo: não pode estar vazio") numa única
        ↪      mensagem
        String mensagem = ex.getBindingResult().getFieldErrors().stream()
            .map(f -> f.getField() + ": " + f.getDefaultMessage())
            .collect(Collectors.joining(", "));
        ErrorResponseDTO erro = new ErrorResponseDTO(mensagem, 400, LocalDateTime.now());
        return ResponseEntity.badRequest().body(erro);
    }

    // Captura qualquer outra exceção não prevista – rede de segurança final,
    // evita vaziar stack trace interna para quem consome a API.
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponseDTO> handleGenerico(Exception ex) {
        ErrorResponseDTO erro = new ErrorResponseDTO("Erro interno inesperado.", 500,
↪      LocalDateTime.now());
        return ResponseEntity.internalServerError().body(erro);
    }
}

```

Testes no Postman

Com techdesk-chamados-api rodando no Terminal 1. Use a mesma Collection TechDesk, numa nova pasta Erros.

1. Erro 404 — chamado inexistente - GET `http://localhost:8080/api/chamados/9999` - Esperado: 404 Not Found, body: `{"mensagem": "Chamado com id 9999 não encontrado.", "status": 404, ...}`

2. Erro 400 — validação - POST `http://localhost:8080/api/chamados`, body:
`{ "titulo": "", "descricao": "", "categoriaId": null }`

- Esperado: 400 Bad Request, mensagem listando os campos inválidos.

Adicione essas duas requisições na mesma collection, numa subpasta Erros.

Checkpoint — o que você deve ter, e como confirmar

- As duas requisições acima retornam exatamente os status e formato de erro descritos — nunca um 500 genérico para esses casos.
- Você sabe explicar por que capturar `Exception.class` por último é importante (é o handler “genérico”, mais específico sempre deve vir antes).

Módulo 7 — Autenticação e Segurança (JWT) + Postman

Objetivo

Proteger a API para que só usuários autenticados criem/vejam chamados.

Teoria (aprofundada)

Spring Security intercepta toda requisição antes de chegar ao Controller e decide se ela pode passar. **JWT (JSON Web Token)** é um token assinado digitalmente que carrega informações do usuário (como o e-mail) e uma validade. Diferente de sessões tradicionais, o servidor **não guarda estado** — cada requisição chega com o token no header `Authorization: Bearer <token>`, e o servidor só precisa validar a assinatura para confiar no conteúdo. Isso é o que torna JWT ideal para arquiteturas de microsserviços: qualquer serviço com a mesma chave secreta consegue validar o token, sem depender de um banco de sessões compartilhado.

Passo 1 — Adicionar as dependências de segurança

Pare a aplicação (Ctrl+C no Terminal 1) e adicione ao `pom.xml` do `techdesk-chamados-api`:

```
<!-- techdesk-chamados-api/pom.xml (dentro de <dependencies>) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>com.auth0</groupId>
  <artifactId>java-jwt</artifactId>
  <version>4.4.0</version>
</dependency>
```

A partir do momento em que `spring-boot-starter-security` entra no classpath, o Spring **bloqueia todos os endpoints por padrão**, inclusive `/api/status` — é só com o `SecurityConfig` (adiante) que voltamos a liberar o que precisa ser público. Se você rodar a aplicação entre adicionar essa dependência e terminar o `SecurityConfig`, é esperado ver 401 em tudo, temporariamente.

No terminal, dentro de `techdesk/techdesk-chamados-api/`, rode `mvn clean install` (ou deixe a IDE reimportar) antes de continuar.

```
// techdesk-chamados-api/src/main/java/com/techdesk/chamados/service/TokenService.java
```

```
@Service
public class TokenService {

    @Value("${jwt.secret}")
    private String secret;

    // Gera o token assinado, contendo o email do usuário e o prazo de expiração.
    public String gerarToken(Usuario usuario) {
        Algorithm algoritmo = Algorithm.HMAC256(secret);
        return JWT.create()
            .withIssuer("techdesk-chamados-api")
            .withSubject(usuario.getEmail())
            .withExpiresAt(Instant.now().plus(2, ChronoUnit.HOURS))
            .sign(algoritmo);
    }

    // Valida o token recebido e retorna o email do dono, se for válido.
    public String validarToken(String token) {
        Algorithm algoritmo = Algorithm.HMAC256(secret);
        return JWT.require(algoritmo)
            .withIssuer("techdesk-chamados-api")
            .build()
            .verify(token)
            .getSubject();
    }
}
```

```

    }
}

// techdesk-chamados-
↪ api/src/main/java/com/techdesk/chamados/security/AutenticacaoFilter.java

// Filtro executado em TODA requisição, antes do Controller:
// lê o header Authorization, valida o token e autentica o usuário no contexto do Spring.
@Component
public class AutenticacaoFilter extends OncePerRequestFilter {

    private final TokenService tokenService;
    private final UsuarioRepository usuarioRepository;

    public AutenticacaoFilter(TokenService tokenService, UsuarioRepository
    ↪ usuarioRepository) {
        this.tokenService = tokenService;
        this.usuarioRepository = usuarioRepository;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse
    ↪ response,
                                FilterChain filterChain) throws ServletException,
    ↪ IOException {
        String header = request.getHeader("Authorization");
        if (header != null) {
            String token = header.replace("Bearer ", "");
            String email = tokenService.validarToken(token);
            Usuario usuario = usuarioRepository.findByEmail(email).orElse(null);
            if (usuario != null) {
                var auth = new UsernamePasswordAuthenticationToken(usuario, null,
    ↪ List.of());
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
            filterChain.doFilter(request, response); // sempre continua a cadeia de filtros
        }
    }
}

// techdesk-chamados-api/src/main/java/com/techdesk/chamados/config/SecurityConfig.java

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final AutenticacaoFilter autenticacaoFilter;

    public SecurityConfig(AutenticacaoFilter autenticacaoFilter) {
        this.autenticacaoFilter = autenticacaoFilter;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable()) // desnecessário para API stateless com JWT
            .sessionManagement(s ->
    ↪ s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**", "/api/status").permitAll() //
    ↪ login/status liberados

```

```

        .anyRequest().authenticated() //
↪ resto exige token válido
    )
    .addFilterBefore(authenticacaoFilter,
↪ UsernamePasswordAuthenticationFilter.class)
    .build();
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(); // nunca salvar senha em texto puro
}
}

```

(O *AuthController* com */api/auth/registrar* e */api/auth/login* segue o mesmo padrão *Controller* → *Service* → *Repository* já usado em *Chamado* — crie em *controller/AuthController.java*.)

Reinicie a aplicação no **Terminal 1** (`mvn spring-boot:run`, dentro de `techdesk/techdesk-chamados-api/`) antes de testar.

Testes no Postman

Use a mesma Collection TechDesk do Postman, numa nova pasta Auth.

1. Registrar usuário - POST `http://localhost:8080/api/auth/registrar`

```
{ "nome": "Elimar", "email": "elimar@teste.com", "senha": "senha123" }
```

- Esperado: 201 Created.

2. Login - POST `http://localhost:8080/api/auth/login`

```
{ "email": "elimar@teste.com", "senha": "senha123" }
```

- Esperado: 200 OK, body com o token. **Copie esse token.**

3. Acesso sem token - GET `http://localhost:8080/api/chamados` (sem header Authorization) - Esperado: 401 Unauthorized ou 403 Forbidden.

4. Acesso com token - Mesma requisição, mas em Headers adicione: Authorization: Bearer <token copiado> - Esperado: 200 OK, lista de chamados normalmente.

Dica de Postman: salve o token numa **variável de ambiente** (`{{token}}`) depois do login, usando a aba “Tests” da requisição de login com o script `pm.environment.set("token", pm.response.json().token);`. Assim as próximas requisições usam Authorization: Bearer `{{token}}` automaticamente.

Checkpoint — o que você deve ter, e como confirmar

- Sem token, qualquer chamada a `/api/chamados` é bloqueada com 401/403.
- Com token válido, as chamadas funcionam normalmente.
- Você entende por que JWT não exige que o servidor “lembre” quem está logado (stateless).

Módulo 8 — Testes Automatizados (JUnit + Mockito)

Objetivo

Substituir os testes manuais/hardcoded por testes automatizados — a lacuna técnica que você identificou no diagnóstico.

Teoria (aprofundada)

Teste unitário: testa uma classe isolada, “mockando” (simulando) suas dependências, para garantir que a lógica interna está correta sem depender de banco de dados real ou rede. **Teste de integração:** sobe o contexto real (ou parte dele) e testa o comportamento de ponta a ponta, incluindo a integração entre camadas. Os dois são complementares: testes unitários são rápidos e você escreve muitos; testes de integração são mais lentos e você escreve menos, focados nos fluxos críticos.

```
// techdesk-chamados-
↪ api/src/test/java/com/techdesk/chamados/service/ChamadoServiceTest.java

@ExtendWith(MockitoExtension.class)
class ChamadoServiceTest {

    @Mock
    private ChamadoRepository chamadoRepository; // dependência "falsa", controlada pelo
    ↪ teste

    @Mock
    private CategoriaRepository categoriaRepository;

    @InjectMocks
    private ChamadoService chamadoService; // instância real, com os mocks acima injetados

    @Test
    void deveCriarChamadoComSucesso() {
        // Arrange: prepara o cenário do teste
        Categoria categoria = new Categoria();
        categoria.setId(1L);
        categoria.setNome("Hardware");

        ChamadoRequestDTO dto = new ChamadoRequestDTO("Impressora não liga",
    ↪ "Detalhes...", 1L);
        Usuario usuario = new Usuario();

        when(categoriaRepository.findById(1L)).thenReturn(Optional.of(categoria));
        when(chamadoRepository.save(any(Chamado.class))).thenAnswer(inv ->
    ↪ inv.getArgument(0));

        // Act: executa o método testado
        ChamadoResponseDTO resultado = chamadoService.criar(dto, usuario);

        // Assert: verifica se o resultado é o esperado
        assertEquals("Impressora não liga", resultado.titulo());
        assertEquals(StatusChamado.ABERTO, resultado.status());
        verify(chamadoRepository, times(1)).save(any(Chamado.class)); // confirma que save
    ↪ foi chamado 1x
    }

    @Test
    void deveLancarExcecaoQuandoChamadoNaoExiste() {
        when(chamadoRepository.findById(99L)).thenReturn(Optional.empty());

        assertThrows(ChamadoNaoEncontradoException.class,
            () -> chamadoService.buscarPorId(99L));
    }
}
```

```
// techdesk-chamados-
↪ api/src/test/java/com/techdesk/chamados/controller/ChamadoControllerIT.java

// Teste de integração: sobe o contexto real do Spring e testa o endpoint por fora,
// simulando uma requisição HTTP de verdade.
@SpringBootTest
@AutoConfigureMockMvc
class ChamadoControllerIT {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void deveRetornar401SemToken() throws Exception {
        mockMvc.perform(get("/api/chamados"))
            .andExpect(status().isUnauthorized());
    }
}

mvn test
git add .
git commit -m "test(chamados-api): testes unitários e de integração com JUnit e Mockito"
git push
```

Checkpoint — o que você deve ter, e como confirmar

- `mvn test` roda e mostra BUILD SUCCESS, com todos os testes passando (verifique o resumo Tests run: X, Failures: 0).
- Você consegue explicar, com suas palavras, por que `@Mock` simula uma dependência e `@InjectMocks` injeta esses mocks na classe testada.
- Se um teste falhar, leia a mensagem do assert antes de mexer no código — ela diz exatamente o que era esperado vs. o que veio.

Módulo 9 — Segundo Microserviço: techdesk-notificacao-api

Objetivo

Criar um segundo serviço, independente, responsável por registrar e consultar notificações relacionadas aos chamados.

Teoria (aprofundada)

Um microserviço é uma aplicação **independente**, com seu próprio processo, seu próprio banco de dados (ou schema) e seu próprio ciclo de deploy. A regra mais importante: **cada serviço é dono dos seus dados** — o techdesk-notificacao-api nunca acessa o banco do techdesk-chamados-api diretamente; ele só recebe informação através de uma API ou de um evento. Isso é o que permite que os dois evoluam, escalem e até quebrem de forma independente, sem um travar o outro.

Estrutura de pacotes do techdesk-notificacao-api

```
com.techdesk.notificacao
├── NotificacaoApiApplication.java
├── config/
```



```

├── RabbitMQConfig.java          # Módulo 11
├── controller/
│   └── NotificacaoController.java
├── dto/
│   └── NotificacaoResponseDTO.java
├── entity/
│   └── Notificacao.java
├── repository/
│   └── NotificacaoRepository.java
├── service/
│   └── NotificacaoService.java
├── messaging/                  # Módulo 11
│   ├── ChamadoCriadoEvent.java
│   └── ChamadoEventListener.java

```

Passo a passo

1. Novo projeto no start.spring.io: artifactId: techdesk-notificacao-api, pacote com.techdesk.notificacao, dependências Spring Web, Spring Data JPA, MySQL Driver, Validation, DevTools. (Spring AMQP fica de fora por enquanto — só entra no Módulo 11, quando for usado, seguindo a mesma regra do Módulo 3.)
2. Extraia de forma que a pasta gerada fique em techdesk/techdesk-notificacao-api/ (irmã de techdesk-chamados-api/, dentro de techdesk/).
3. Adicione um segundo banco no docker-compose.yml **da raiz** (abra techdesk/docker-compose.yml — o mesmo arquivo do Módulo 0), para que cada serviço tenha seu próprio schema:

```

# trecho a adicionar dentro de "services:" em techdesk/docker-compose.yml
mysql-notificacao:
  image: mysql:8.0
  container_name: techdesk-mysql-notificacao
  environment:
    MYSQL_ROOT_PASSWORD: techdesk123
    MYSQL_DATABASE: techdesk_notificacao
  ports:
    - "3307:3306"

```

Depois de salvar, aplique a mudança: abra um terminal **dentro de techdesk/** (a raiz, onde está o docker-compose.yml) e rode:

```
docker-compose up -d
```

Isso sobe só o novo container techdesk-mysql-notificacao, sem afetar o techdesk-mysql e o techdesk-rabbitmq que já estavam rodando. Confirme com `docker ps` — devem aparecer 3 containers agora.

```

// techdesk-notificacao-
↪ api/src/main/java/com/techdesk/notificacao/entity/Notificacao.java

```

```

@Entity
@Table(name = "notificacoes")
public class Notificacao {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long chamadoId;    // referência ao chamado de OUTRO serviço — só o id,
    ↪ nunca o objeto inteiro

```

```

    private String tituloChamado;
    private String mensagem;
    private LocalDateTime dataEnvio;

    public Notificacao() {}
    // getters e setters – gere pela IDE
}

// techdesk-notificacao-
↪ api/src/main/java/com/techdesk/notificacao/repository/NotificacaoRepository.java
public interface NotificacaoRepository extends JpaRepository<Notificacao, Long> {
    List<Notificacao> findByChamadoId(Long chamadoId);
}

// techdesk-notificacao-
↪ api/src/main/java/com/techdesk/notificacao/service/NotificacaoService.java
@Service
public class NotificacaoService {

    private final NotificacaoRepository repository;

    public NotificacaoService(NotificacaoRepository repository) {
        this.repository = repository;
    }

    public Notificacao registrar(Long chamadoId, String tituloChamado, String mensagem) {
        Notificacao notificacao = new Notificacao();
        notificacao.setChamadoId(chamadoId);
        notificacao.setTituloChamado(tituloChamado);
        notificacao.setMensagem(mensagem);
        notificacao.setDataEnvio(LocalDateTime.now());
        return repository.save(notificacao);
    }

    public List<Notificacao> listarPorChamado(Long chamadoId) {
        return repository.findByChamadoId(chamadoId);
    }
}

// techdesk-notificacao-
↪ api/src/main/java/com/techdesk/notificacao/controller/NotificacaoController.java
@RestController
@RequestMapping("/api/notificacoes")
public class NotificacaoController {

    private final NotificacaoService service;

    public NotificacaoController(NotificacaoService service) {
        this.service = service;
    }

    @GetMapping("/chamado/{chamadoId}")
    public ResponseEntity<List<Notificacao>> listarPorChamado(@PathVariable Long
        ↪ chamadoId) {
        return ResponseEntity.ok(service.listarPorChamado(chamadoId));
    }
}

```

```
# techdesk-notificacao-api/src/main/resources/application.yml
spring:
  application:
    name: techdesk-notificacao-api
  datasource:
    url: jdbc:mysql://localhost:3307/techdesk_notificacao
    username: root
    password: techdesk123
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  server:
    port: 8081 # porta diferente do chamados-api, que já usa 8080
```

Agora suba os **dois serviços ao mesmo tempo**, cada um no seu próprio terminal:

```
# Terminal 1 – dentro de techdesk/techdesk-chamados-api/ (se não estiver rodando ainda)
mvn spring-boot:run
```

```
# Terminal 2 – dentro de techdesk/techdesk-notificacao-api/
mvn spring-boot:run
```

A partir de agora, vamos nos referir a esses dois terminais como **Terminal 1** (chamados-api, porta 8080) e **Terminal 2** (notificacao-api, porta 8081) pelo resto do curso.

```
git add techdesk-notificacao-api/
git commit -m "feat(notificacao-api): estrutura inicial do segundo microsserviço"
git push
```

Checkpoint — o que você deve ter, e como confirmar

- Os dois serviços sobem **ao mesmo tempo**, em portas diferentes (8080 e 8081), sem conflito — Terminal 1 e Terminal 2 rodando lado a lado.
- GET `http://localhost:8081/api/notificacoes/chamado/1` responde 200 OK com uma lista vazia (ainda não há notificações — vamos gerar no próximo módulo).
- Você sabe explicar por que a Notificacao guarda só o chamadoId (um número), e não uma referência direta à Entity Chamado do outro serviço.

Módulo 10 — Comunicação Síncrona entre os Dois Serviços (REST)

Objetivo

Fazer o chamados-api chamar o notificacao-api via REST toda vez que um chamado é criado — e sentir na prática o problema que essa abordagem gera.

Teoria

A forma mais simples de um serviço “consumir” outro é fazer uma chamada HTTP direta, com um cliente como o RestClient (substituto moderno do antigo RestTemplate). Funciona, mas cria um problema real: se o notificacao-api estiver fora do ar, lento, ou com erro, a criação do chamado — que é a operação principal — **também falha ou fica lenta**, mesmo sendo uma consequência secundária. Esse é o problema clássico de **acoplamento forte entre serviços**, e é exatamente o que o Módulo 11 resolve.

```
// techdesk-chamados-
↪ api/src/main/java/com/techdesk/chamados/config/RestClientConfig.java

@Configuration
public class RestClientConfig {

    @Bean
    public RestClient notificacaoRestClient() {
        return RestClient.builder()
            .baseUrl("http://localhost:8081") // endereço do notificacao-api
            .build();
    }
}
```

```
// techdesk-chamados-
↪ api/src/main/java/com/techdesk/chamados/service/NotificacaoClient.java

// Classe dedicada a se comunicar com o OUTRO serviço – mantém essa
// responsabilidade isolada, fora do ChamadoService.
@Service
public class NotificacaoClient {

    private final RestClient restClient;

    public NotificacaoClient(RestClient notificacaoRestClient) {
        this.restClient = notificacaoRestClient;
    }

    public void notificar(Long chamadoId, String titulo) {
        restClient.post()
            .uri("/api/notificacoes")
            .contentType(MediaType.APPLICATION_JSON)
            .body(Map.of(
                "chamadoId", chamadoId,
                "tituloChamado", titulo,
                "mensagem", "Novo chamado aberto: " + titulo
            ))
            .retrieve()
            .toBodilessEntity();
    }
}
```

(Adicione também um POST /api/notificacoes no NotificacaoController do outro serviço, chamando service.registrar(...), para receber essa chamada.)

```
// No ChamadoService, injete o NotificacaoClient e chame no final de criar():
public ChamadoResponseDTO criar(ChamadoRequestDTO dto, Usuario usuarioLogado) {
    // ... lógica existente de criação ...
    Chamado salvo = chamadoRepository.save(chamado);

    notificacaoClient.notificar(salvo.getId(), salvo.getTitulo()); // chamada síncrona ao
    ↪ outro serviço

    return toResponseDTO(salvo);
}
```

Teste no Postman

1. Com os dois serviços rodando (Terminal 1 e Terminal 2), crie um chamado normalmente pela Collection do Postman (POST /api/chamados, Módulo 5).
2. Em seguida, no Postman, faça GET `http://localhost:8081/api/notificacoes/chamado/{id}` — deve retornar a notificação criada automaticamente.
3. **Agora pare o notificacao-api:** clique no **Terminal 2** e pressione Ctrl+C. Com ele parado, tente criar outro chamado pelo Postman (POST /api/chamados, Terminal 1 continua rodando). Observe: a criação do chamado falha ou demora, mesmo o problema sendo só no serviço de notificação — essa é a prova prática do acoplamento. Depois, suba o Terminal 2 de novo (`mvn spring-boot:run`) para seguir para o próximo módulo.

Checkpoint — o que você deve ter, e como confirmar

- Criar um chamado gera automaticamente uma notificação no outro serviço, visível via GET.
- Você reproduziu o cenário de falha (derrubando o notificacao-api) e viu o chamados-api ser afetado — guarde essa observação, ela é a motivação real do próximo módulo.

Módulo 11 — Comunicação Assíncrona e Desacoplada (RabbitMQ)

Objetivo

Refatorar a comunicação entre os dois serviços para ser **assíncrona, confiável e com baixo acoplamento**, usando fila de mensagens — o objetivo final do projeto.

Teoria (aprofundada)

Em vez do chamados-api chamar diretamente o notificacao-api, ele passa a **publicar um evento** numa fila do RabbitMQ e seguir seu fluxo normalmente, sem esperar resposta. O notificacao-api **escuta** essa fila de forma independente e processa quando conseguir. Isso resolve os três requisitos:

- **Assíncrona:** o chamados-api não espera o notificacao-api processar nada — ele publica a mensagem e continua.
- **Confiável:** o RabbitMQ guarda a mensagem numa fila **durável** até que o consumidor a processe com sucesso (ack). Se o notificacao-api estiver fora do ar, a mensagem fica esperando na fila — nada se perde. Se o processamento falhar, a mensagem pode ser reencaminhada para uma **Dead Letter Queue (DLQ)**, uma fila separada para mensagens com problema, em vez de sumir silenciosamente.
- **Baixo acoplamento:** o chamados-api não sabe (e não precisa saber) que o notificacao-api existe. Ele só conhece o *contrato do evento* (o formato da mensagem) e a fila. Amanhã, um terceiro serviço poderia escutar a mesma fila sem que o chamados-api mude uma linha de código.

Isso usa o **RabbitMQ** já rodando desde o Módulo 0. O painel `localhost:15672` já deveria estar acessível — se não estiver, volte para a pasta `techdesk/` e rode `docker-compose up -d` antes de continuar. O padrão usado é **Exchange → Fila → Consumidor**: o chamados-api publica num *exchange*, o RabbitMQ roteia para a *fila* de acordo com uma *routing key*, e o notificacao-api consome dessa fila.

Passo 1 — Adicionar a dependência de mensageria (nos dois projetos)

Pare os dois serviços (Ctrl+C no Terminal 1 e no Terminal 2) e adicione, **em cada um dos dois pom.xml** (`techdesk-chamados-api/pom.xml` e `techdesk-notificacao-api/pom.xml`):

```

<!-- dentro de <dependencies>, nos DOIS projetos -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

```

Rode `mvn clean install` em cada um (dois terminais separados, um por pasta) antes de seguir para o código.

Passo 2 — Configuração no chamados-api (publicador)

```
// techdesk-chamados-api/src/main/java/com/techdesk/chamados/config/RabbitMQConfig.java
```

```

@Configuration
public class RabbitMQConfig {

    public static final String EXCHANGE = "techdesk.exchange";
    public static final String ROUTING_KEY_CHAMADO_CRIADO = "chamado.criado";

    @Bean
    public TopicExchange exchange() {
        return new TopicExchange(EXCHANGE);
    }

    @Bean
    public MessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter(); // converte objetos Java <-> JSON
        ↪ automaticamente
    }

    @Bean
    public AmqpTemplate amqpTemplate(ConnectionFactory connectionFactory,
        ↪ MessageConverter converter) {
        RabbitTemplate template = new RabbitTemplate(connectionFactory);
        template.setMessageConverter(converter);
        return template;
    }
}

```

```

// techdesk-chamados-
↪ api/src/main/java/com/techdesk/chamados/messaging/ChamadoCriadoEvent.java

```

```

// O "contrato" da mensagem – o formato que qualquer serviço interessado precisa conhecer.
// Como é um record simples, é fácil replicar no outro serviço sem criar dependência
↪ direta.
public record ChamadoCriadoEvent(
    Long chamadoId,
    String titulo,
    String categoriaNome,
    LocalDateTime dataAbertura
) {}

```

```

// techdesk-chamados-
↪ api/src/main/java/com/techdesk/chamados/messaging/ChamadoEventPublisher.java

```

```

@Component
public class ChamadoEventPublisher {

    private final AmqpTemplate amqpTemplate;

```

```

public ChamadoEventPublisher(AmqpTemplate amqpTemplate) {
    this.amqpTemplate = amqpTemplate;
}

public void publicarChamadoCriado(Chamado chamado) {
    ChamadoCriadoEvent evento = new ChamadoCriadoEvent(
        chamado.getId(), chamado.getTitulo(),
        chamado.getCategoria().getNome(), chamado.getDataAbertura()
    );
    // Publica no exchange, com a routing key – o Rabbit cuida do resto.
    // Isso NÃO espera resposta de ninguém: é "fire and forget" confiável.
    amqpTemplate.convertAndSend(
        RabbitMQConfig.EXCHANGE,
        RabbitMQConfig.ROUTING_KEY_CHAMADO_CRIADO,
        evento
    );
}
}

// No ChamadoService, troque a chamada síncrona pelo publisher de evento:
public ChamadoResponseDTO criar(ChamadoRequestDTO dto, Usuario usuarioLogado) {
    // ... lógica existente de criação ...
    Chamado salvo = chamadoRepository.save(chamado);

    chamadoEventPublisher.publicarChamadoCriado(salvo); // assíncrono – não bloqueia, não
    ↳ derruba nada

    return toResponseDTO(salvo);
}

```

Configuração no notificacao-api (consumidor)

```

// techdesk-notificacao-
↳ api/src/main/java/com/techdesk/notificacao/config/RabbitMQConfig.java

@Configuration
public class RabbitMQConfig {

    public static final String EXCHANGE = "techdesk.exchange";
    public static final String FILA_CHAMADO_CRIADO = "notificacao.chamado.criado";
    public static final String ROUTING_KEY = "chamado.criado";
    public static final String DLQ = "notificacao.chamado.criado.dlq"; // Dead Letter Queue

    @Bean
    public TopicExchange exchange() {
        return new TopicExchange(EXCHANGE);
    }

    @Bean
    public Queue filaChamadoCriado() {
        // Fila DURÁVEL (sobrevive a reinício do RabbitMQ) e com DLQ configurada:
        // mensagens que falham repetidamente vão para a DLQ, em vez de se perderem.
        return QueueBuilder.durable(FILA_CHAMADO_CRIADO)
            .withArgument("x-dead-letter-exchange", "")
            .withArgument("x-dead-letter-routing-key", DLQ)
            .build();
    }
}

```

```

@Bean
public Queue dlq() {
    return QueueBuilder.durable(DLQ).build();
}

@Bean
public Binding binding(Queue filaChamadoCriado, TopicExchange exchange) {
    return BindingBuilder.bind(filaChamadoCriado).to(exchange).with(ROUTING_KEY);
}

@Bean
public MessageConverter jsonMessageConverter() {
    return new Jackson2JsonMessageConverter();
}
}

```

```

// techdesk-notificacao-
↪ api/src/main/java/com/techdesk/notificacao/messaging/ChamadoCriadoEvent.java

```

```

// Mesmo formato do evento publicado pelo chamados-api – cada serviço tem
// sua PRÓPRIA cópia do contrato (não compartilham código Java diretamente,
// só o formato JSON), o que preserva a independência entre os dois.

```

```

public record ChamadoCriadoEvent(
    Long chamadoId,
    String titulo,
    String categoriaNome,
    LocalDateTime dataAbertura
) {}

```

```

// techdesk-notificacao-
↪ api/src/main/java/com/techdesk/notificacao/messaging/ChamadoEventListener.java

```

```

@Component
public class ChamadoEventListener {

    private final NotificacaoService notificacaoService;

    public ChamadoEventListener(NotificacaoService notificacaoService) {
        this.notificacaoService = notificacaoService;
    }

    // @RabbitListener escuta a fila continuamente, em background,
    // sem que ninguém precise "chamar" este método manualmente.
    @RabbitListener(queues = RabbitMQConfig.FILA_CHAMADO_CRIADO)
    public void ouvirChamadoCriado(ChamadoCriadoEvent evento) {
        notificacaoService.registrar(
            evento.chamadoId(),
            evento.titulo(),
            "Novo chamado aberto: " + evento.titulo() + " (categoria: " +
            ↪ evento.categoriaNome() + ")"
        );
        // Se este método terminar sem lançar exceção, o Rabbit confirma (ack)
        // automaticamente e remove a mensagem da fila. Se lançar exceção,
        // a mensagem é reencaminhada (e, após tentativas, vai para a DLQ).
    }
}

```

```

# adicionar em application.yml dos DOIS serviços
spring:

```



```
rabbitmq:
  host: localhost
  port: 5672
  username: guest
  password: guest
```

Teste no Postman + RabbitMQ

1. Suba os dois serviços de novo: `mvn spring-boot:run` no **Terminal 1** (dentro de `techdesk-chamados-api/`) e no **Terminal 2** (dentro de `techdesk-notificacao-api/`).
2. No Postman, crie um chamado (POST `/api/chamados`, Módulo 5).
3. Confira em `http://localhost:8081/api/notificacoes/chamado/{id}` — a notificação aparece, agora criada de forma assíncrona.
4. **Repita o teste de falha do Módulo 10:** pare o `notificacao-api` (Ctrl+C no Terminal 2) e crie outro chamado pelo Postman. Desta vez, a criação do chamado funciona normalmente (201 Created sem demora) — a mensagem fica esperando na fila.
5. Suba o Terminal 2 novamente (`mvn spring-boot:run`). Sem fazer nada manualmente, a mensagem pendente é processada e a notificação aparece.
6. Acesse `http://localhost:15672` no navegador (painel do RabbitMQ) → aba **Queues** → confirme visualmente a fila `notificacao.chamado.criado` e observe mensagens acumulando quando o consumidor está fora do ar.

```
git add .
git commit -m "feat: comunicação assíncrona entre chamados-api e notificacao-api via
↳ RabbitMQ"
git push
```

Checkpoint — o que você deve ter, e como confirmar

- Criar um chamado com o `notificacao-api` **fora do ar** não trava nem falha a criação — essa é a prova de que o acoplamento foi removido.
- Ao religar o `notificacao-api`, as mensagens pendentes são processadas automaticamente, sem intervenção manual.
- Você consegue explicar, em uma frase cada, o que torna essa comunicação assíncrona, confiável e de baixo acoplamento — essa explicação, sozinha, é uma resposta forte de entrevista técnica de nível pleno.

Módulo 12 — Frontend Consumindo a API

Objetivo

Criar uma tela simples para abrir e listar chamados, dando um rosto visual ao projeto.

Teoria

O objetivo aqui não é aprender um framework front-end novo, é provar visualmente que o backend funciona de ponta a ponta. HTML + CSS + JavaScript puro, com `fetch`, já cumpre esse papel — inclusive fica mais fácil de rodar sem build tools, o que facilita demonstração.

Atenção a um detalhe que quebra o teste se pulado: o navegador bloqueia por padrão requisições `fetch` de uma origem para outra (CORS) — e um frontend aberto direto como arquivo (`file://`) ou servido em `localhost:5500` é, para o navegador, uma origem diferente de `localhost:8080`. Sem liberar isso no backend, o `fetch` do

frontend falha silenciosamente. Resolvemos isso com uma configuração de CORS no chamados-api, feita abaixo.

Passo 1 — Liberar CORS no chamados-api

Pare o Terminal 1 e adicione esta classe:

```
// techdesk-chamados-api/src/main/java/com/techdesk/chamados/config/CorsConfig.java

@Configuration
public class CorsConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                // Libera o frontend (rodando na porta 5500, ver Passo 3) a chamar a API
                registry.addMapping("/api/**")
                    .allowedOrigins("http://localhost:5500", "http://127.0.0.1:5500")
                    .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE")
                    .allowedHeaders("*");
            }
        };
    }
}
```

Suba o Terminal 1 de novo (mvn spring-boot:run) antes de testar o frontend.

Passo 2 — Criar as páginas do frontend

Crie a pasta frontend/ dentro de techdesk/ (irmã de techdesk-chamados-api/), com os três arquivos abaixo:

```
<!-- frontend/index.html -->
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <title>TechDesk – Painel de Chamados</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>TechDesk</h1>

    <form id="form-chamado">
        <input type="text" id="titulo" placeholder="Título do chamado" required>
        <textarea id="descricao" placeholder="Descreva o problema" required></textarea>
        <button type="submit">Abrir chamado</button>
    </form>

    <table id="tabela-chamados">
        <thead>
            <tr><th>Título</th><th>Status</th><th>Categoria</th></tr>
        </thead>
        <tbody></tbody>
    </table>

    <script src="app.js"></script>
```

```
</body>
</html>
```

```
// frontend/app.js
const API_URL = "http://localhost:8080/api/chamados";
const token = localStorage.getItem("token"); // obtido após login (Módulo 7)

// Busca os chamados na API e monta a tabela
async function carregarChamados() {
  const resposta = await fetch(API_URL, {
    headers: { "Authorization": `Bearer ${token}` }
  });
  const chamados = await resposta.json();

  const corpoTabela = document.querySelector("#tabela-chamados tbody");
  corpoTabela.innerHTML = "";

  chamados.forEach(chamado => {
    const linha = document.createElement("tr");
    linha.innerHTML = `
      <td>${chamado.titulo}</td>
      <td>${chamado.status}</td>
      <td>${chamado.categoriaNome}</td>
    `;
    corpoTabela.appendChild(linha);
  });
}

// Envia um novo chamado para a API
document.getElementById("form-chamado").addEventListener("submit", async (evento) => {
  evento.preventDefault();

  const novoChamado = {
    titulo: document.getElementById("titulo").value,
    descricao: document.getElementById("descricao").value,
    categoriaId: 1
  };

  await fetch(API_URL, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Authorization": `Bearer ${token}`
    },
    body: JSON.stringify(novoChamado)
  });

  document.getElementById("form-chamado").reset();
  carregarChamados(); // recarrega a lista após criar
});

carregarChamados();

/* frontend/style.css – o mínimo para ficar apresentável em print/demo */
body { font-family: system-ui, sans-serif; max-width: 800px; margin: 40px auto;
  ↪ background: #f4f6f8; }
h1 { color: #1F4E79; }
form { display: flex; flex-direction: column; gap: 8px; margin-bottom: 24px; }
input, textarea, button { padding: 10px; border-radius: 6px; border: 1px solid #ccc; }
button { background: #2E75B6; color: white; border: none; cursor: pointer; }
```

```
table { width: 100%; border-collapse: collapse; background: white; }
th, td { padding: 10px; border-bottom: 1px solid #eee; text-align: left; }
```

Passo 3 — Conseguir um token para o frontend usar

O app.js lê o token de `localStorage.getItem("token")`, mas a tela não tem formulário de login (fora do escopo deste projeto simples). Para testar:

1. No Postman, faça o login (POST /api/auth/login, Módulo 7) e copie o token da resposta.
2. Abra o frontend no navegador, abra o **DevTools** (F12) → aba **Console**, e rode:

```
localStorage.setItem("token", "COLE_O_TOKEN_AQUI");
```

3. Recarregue a página.

Passo 4 — Servir o frontend (não abra o arquivo direto)

Abrir `index.html` clicando duas vezes (URL `file:///...`) faz o navegador tratar a página como “origem nula”, e o fetch para a API é bloqueado mesmo com CORS configurado. Sirva com um servidor local simples: abra um terminal **dentro de techdesk/frontend/** e rode:

```
python3 -m http.server 5500
```

(ou, no VS Code, clique com o botão direito em `index.html` → “Open with Live Server”, se tiver a extensão — também sobe na porta 5500 por padrão). Acesse `http://localhost:5500` no navegador — **não** `file:///`.

```
git add frontend/
git commit -m "feat(frontend): tela de listagem e criação de chamados"
git push
```

Checkpoint — o que você deve ter, e como confirmar

- Acessando `http://localhost:5500` (servido pelo Passo 4, com token salvo no Passo 3, e o `chamados-api` rodando no Terminal 1), a tabela carrega os chamados existentes sem erro de CORS no Console.
- Criar um chamado pelo formulário atualiza a tabela sem precisar recarregar a página manualmente.
- Isso é o que você vai gravar em vídeo curto ou capturar em print para o README e o LinkedIn.

Módulo 13 — Docker Compose Multi-Serviço (Aprofundamento)

Objetivo

Empacotar os dois microsserviços, os dois bancos e o RabbitMQ num único `docker-compose.yml`, para subir o projeto inteiro com um comando.

Passo 1 — Encerrar os containers “manuais” antes de trocar o arquivo

Os containers do Módulo 0 e do Módulo 9 (`techdesk-mysql`, `techdesk-rabbitmq`, `techdesk-mysql-notificacao`) ainda usam as portas 3306, 3307 e 5672/15672. Antes de substituir o `docker-compose.yml`, pare-os para não haver conflito de porta com a versão nova. No terminal, **dentro de techdesk/** (com o `docker-compose.yml` antigo ainda no lugar):

```
docker-compose down
```

Feche também os Terminais 1 e 2 (Ctrl+C) — a partir de agora eles não vão mais rodar via mvn spring-boot:run diretamente, e sim dentro dos containers.

Passo 2 — Criar os Dockerfiles de cada serviço

```
# techdesk-chamados-api/Dockerfile
FROM eclipse-temurin:21-jdk-alpine AS build
WORKDIR /app
COPY . .
RUN ./mvnw clean package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

# techdesk-notificacao-api/Dockerfile (idêntico, porta 8081)
FROM eclipse-temurin:21-jdk-alpine AS build
WORKDIR /app
COPY . .
RUN ./mvnw clean package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Passo 3 — Substituir o docker-compose.yml da raiz

Abra techdesk/docker-compose.yml e **substitua todo o conteúdo** pela versão abaixo:

```
# techdesk/docker-compose.yml (versão final, substitui a do Módulo 0)
version: '3.8'

services:
  mysql-chamados:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: techdesk123
      MYSQL_DATABASE: techdesk_chamados
    ports: ["3306:3306"]
    volumes: ["mysql-chamados-data:/var/lib/mysql"]

  mysql-notificacao:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: techdesk123
      MYSQL_DATABASE: techdesk_notificacao
    ports: ["3307:3306"]
    volumes: ["mysql-notificacao-data:/var/lib/mysql"]

  rabbitmq:
    image: rabbitmq:3-management
    ports: ["5672:5672", "15672:15672"]

  chamados-api:
    build: ./techdesk-chamados-api
```

```

depends_on: [mysql-chamados, rabbitmq]
environment:
  SPRING_DATASOURCE_URL: jdbc:mysql://mysql-chamados:3306/techdesk_chamados
  SPRING_RABBITMQ_HOST: rabbitmq
ports: ["8080:8080"]

notificacao-api:
  build: ./techdesk-notificacao-api
  depends_on: [mysql-notificacao, rabbitmq]
  environment:
    SPRING_DATASOURCE_URL: jdbc:mysql://mysql-notificacao:3306/techdesk_notificacao
    SPRING_RABBITMQ_HOST: rabbitmq
  ports: ["8081:8081"]

volumes:
  mysql-chamados-data:
  mysql-notificacao-data:

```

Passo 4 — Subir tudo com um comando

No terminal, **dentro de techdesk/**:

```
docker-compose up --build
```

Aguarde os 5 containers subirem (o build da imagem Java demora um pouco na primeira vez). Teste os mesmos endpoints do Postman (Módulos 5 a 7) e o frontend (<http://localhost:5500>, servido separadamente como no Módulo 12) — tudo deve funcionar exatamente como antes, agora saindo de containers.

```

git add .
git commit -m "chore: docker-compose multi-serviço completo (2 APIs + 2 bancos +
↳ RabbitMQ)"
git push

```

Checkpoint — o que você deve ter, e como confirmar

- Um único `docker-compose up --build`, rodado do zero numa pasta limpa (ou outra máquina, com Git e Docker instalados — Módulo 0), sobe os 5 containers e o sistema funciona de ponta a ponta — esse é o teste real de que o projeto está reproduzível, não só “funciona na sua máquina”.
- A collection do Postman continua passando nos mesmos testes dos Módulos 5, 6 e 7, agora contra os containers.

Módulo 14 — Publicação, GitHub e Portfólio

Objetivo

Deixar o projeto pronto para ser avaliado por um recrutador ou entrevistador técnico.

Passo a passo

1. Adicione **Swagger/OpenAPI** (`springdoc-openapi-starter-webmvc-ui`) em cada serviço, disponível em `/swagger-ui.html`.
2. Escreva um **README.md** na raiz do repositório, cobrindo: o que o sistema faz e por quê (conecte com sua experiência em suporte técnico), a arquitetura (diagrama simples dos dois

serviços + RabbitMQ + banco), tecnologias usadas, como rodar (docker-compose up), prints do frontend, a collection do Postman, e o que você aprendeu construindo — principalmente sobre acoplamento (Módulo 10 vs. 11).

3. Confirme que o histórico de commits no GitHub está organizado (é o que fizemos módulo a módulo, desde o Módulo 0).
4. No currículo e LinkedIn: *“TechDesk — sistema de gestão de chamados técnicos com arquitetura de microsserviços em Java/Spring Boot, comunicação assíncrona via RabbitMQ, autenticação JWT, testes automatizados e frontend integrado, totalmente containerizado com Docker.”*

Checkpoint final — o que avaliar antes de considerar pronto

- Alguém de fora, só lendo o README e rodando docker-compose up, consegue subir o projeto sem te perguntar nada.
 - Você consegue, em até 2 minutos, explicar em voz alta a arquitetura completa: os dois serviços, por que são separados, e por que a comunicação entre eles é assíncrona — essa é literalmente a pergunta que um entrevistador júnior/pleno faz ao ver esse projeto no currículo.
-

Resumo do que este curso entrega

Dois microsserviços Java/Spring Boot reais, com pacotes organizados seguindo boas práticas, Git e Docker desde o primeiro commit, banco de dados relacional, autenticação JWT, testes automatizados (JUnit/Mockito) com collection Postman documentada, comunicação entre serviços evoluindo de síncrona (REST) para assíncrona e desacoplada (RabbitMQ, com fila durável e DLQ), frontend funcional, e tudo containerizado — cobrindo, de ponta a ponta, a lista de tecnologias exigidas para vagas júnior/pleno de Java em 2026.